Data Preparation and Preprocessing for Intrusion Detection

Devarakonda Saratchandra Mouli
ICT Department
MIT Manipal
Manipal, Karnataka, India

Abstract—

THE PROBLEM….

Intrusion Detection Systems (IDS) are used to detect malicious or suspicious activities in computer network systems. But these systems rely on good quality data. Raw network data usually contains noise, irrelevant information, missing values, and unbalanced classes (for example, way more normal connections than attack ones). This can confuse machine learning models and reduce their accuracy.

To build an effective IDS, it's important to clean and prepare the data properly before using it for training.

This study addresses the challenge of preparing clean, balanced, and normalized datasets to enhance the performance of IDS; specifically, this paper focuses on preparing intrusion data from the KDD Cup 1999 dataset to make it ready for accurate attack detection.

PROPOSED SOLUTION….

We developed and implemented a modular preprocessing pipeline that includes duplicate removal, missing value handling, label encoding for categorical features, and min-max normalization. SMOTE was applied to balance class distributions, particularly targeting underrepresented attack classes. The approach was applied to the 20% subset of the KDD Cup 1999 dataset, preparing a 42-feature dataset suitable for classification.

RESULTS OF THE STUDY…

After preprocessing, the data was used to train Logistic Regression, Decision Tree, and SVM classifiers. The Decision Tree classifier achieved the highest accuracy at 98.70%, followed by SVM and Logistic Regression. Results confirmed improved detection for both frequent and rare attacks due to enhanced data quality and balance.

Keywords—

- ***Data Preprocessing***
- ***Data Cleaning***
- ***Feature Encoding***
- ***Normalization***
- ***Class Imbalance***
- ***SMOTE***
- ***Machine Learning***
- ***Network Security***
- ***KDD Cup 1999***
- ***Classifier Performance***
- ***Feature Selection***
- ***Data Balancing***
- ***Network-based IDS***
- ***Cybersecurity***

1. Introduction

1.1 Problem Statement

In recent years, with the rapid growth of internet-connected systems and the increasing sophistication of cyber threats, network security has become a top priority across industries. **Intrusion Detection Systems (IDS)** are one of the key technologies used to detect and prevent unauthorized access or malicious behaviour in networks. These systems rely heavily on accurate and timely data analysis to classify traffic as normal or intrusive. However, raw network data is often noisy, high-dimensional, and imbalanced, which significantly degrades the performance of machine learning algorithms used in IDS. Key issues include missing values, redundant or irrelevant features, skewed class distribution, and inconsistent formatting. These challenges make it difficult to train reliable models that can effectively detect various types of intrusions, especially rare or sophisticated attacks.

1.2 Proposed System Overview

To address these challenges, this study proposes a comprehensive data preparation and preprocessing framework tailored for intrusion detection systems. The proposed system follows a multi-stage architecture designed to transform raw network traffic data into a form suitable for effective analysis and classification. It starts with data cleaning to handle missing or duplicate records, followed by categorical encoding for converting symbolic attributes such as protocol types and services into numerical values. Next, normalization ensures that all numerical features are on a common scale, preventing features with large numeric ranges from dominating the model. The system also applies data balancing techniques such as **SMOTE (Synthetic Minority Over-sampling Technique**) to correct class imbalances and improve the detection rate of minority intrusions. Finally, the processed data is used to train machine learning classifiers including **Logistic Regression, Decision Trees, and Support Vector Machines(SVM),** with performance evaluated using standard metrics such as **accuracy, precision, recall, and F1-score.**

1.3 Proposed Architecture

The system follows a structured, modular preprocessing and classification pipeline:

1. **Load the Dataset**
   - Import the KDD Cup 1999 dataset (20% subset with 42 features + label).
2. **Clean the Data**
   - Remove duplicate rows.

- Handle missing values by dropping incomplete records.

3. **Encode Categorical Features**
   - Apply Label Encoding to convert symbolic attributes (protocol type, service, flag) into numeric form.
4. **Label Encoding for Output**
   - Encode the label column to convert class names (e.g., "normal", "neptune") into integers.
5. **Drop Low-Sample Classes**
   - Remove classes with fewer than 6 samples to ensure SMOTE compatibility.
6. **Feature Normalization**
   - Use Min-Max Scaling to bring all numerical values into the [0, 1] range.
7. **Class Balancing using SMOTE**
   - Apply SMOTE to generate synthetic samples for underrepresented classes.
8. **Train-Test Split**
   - Divide the balanced dataset into training and test sets (70/30 ratio).
9. **Model Training**
   - Train three classifiers:
     - Logistic Regression
     - Decision Tree
     - Support Vector Machine (SVM)
10. **Model Evaluation**
    - Evaluate model performance using:
      - Accuracy
      - Precision
      - Recall
      - F1-Score

1.4 STRUCTURE OF THE PAPER

The remainder of this paper is organized as follows:

- Section II presents a review of existing literature on data preprocessing for intrusion detection, highlighting current techniques and their limitations.
- Section III explains the design and implementation of the proposed preprocessing pipeline, detailing each stage including encoding, scaling, and balancing.
- Section IV presents the experimental setup, datasets used (such as the KDD Cup 99), and a discussion of results based on performance metrics before and after preprocessing.
- Section V concludes the study, summarizing key findings, the impact of the proposed system, and potential future improvements in IDS preprocessing.

2. MOTIVATION AND STUDY OBJECTIVES

This research is driven by the following key motivations and goals:

1. Improve IDS Performance Through Better Data Quality

Raw or poorly prepared data often leads to low accuracy in detecting threats. This work aims to enhance the performance of IDS models by providing them with clean, well-structured, and balanced data.

2. Develop a Generalized and Repeatable Preprocessing Framework

The study seeks to build a standardized pipeline that can be reused across different intrusion datasets, allowing for consistent and reliable preprocessing that improves the applicability and reproducibility of intrusion detection research.

3. Facilitate Fair Evaluation of Machine Learning Models

By resolving issues such as class imbalance and feature scaling, the proposed system enables a more fair and accurate evaluation of IDS models, ensuring that all types of attacks, including rare ones, are effectively tested and detected.

A few other objectives of this study include:

4. Automate Data Cleaning

Design a repeatable process for identifying and removing duplicate, missing, or irrelevant data.

5. Feature Optimization

Select the most relevant features to reduce dimensionality and improve classifier performance.

6. Boost Detection of Rare Attacks

Use data balancing techniques to improve the recognition of minority class intrusions.

7. Support Real-time Readiness

Prepare data pipelines that can be adapted for use in near real-time intrusion detection systems.

8. Ensure Compatibility with ML Models

Format and scale data to meet the input requirements of various machine learning algorithms.

9. Improve Interpretability

Make the preprocessing steps transparent and easy to understand for reproducibility and further research.

3. LITERATURE SURVEY AND RESEARCH GAPS

Before In the evolving landscape of cybersecurity, the efficacy of Intrusion Detection Systems (IDS) is significantly influenced by the quality of data preprocessing. Numerous studies have delved into various preprocessing techniques to enhance IDS performance. This literature survey examines key contributions in this domain, highlighting the methodologies employed, identified gaps, and areas for future research.

1. Data Curation and Quality Assurance in IDS

Chen et al. [1] conducted an extensive analysis of machine learning-based IDS, emphasizing the pivotal role of data quality. Their study evaluated 11 Host-based Intrusion Detection System (HIDS) datasets across seven machine learning and three deep learning models. Findings revealed that models like BERT and GPT outperformed others, underscoring the influence of data quality on IDS efficacy. However, the research primarily centred on HIDS datasets, suggesting a need for similar evaluations in Network-based Intrusion Detection Systems (NIDS) to generalize the findings.

Gap Identified: The focus on HIDS limits generalizability to network-based IDS (NIDS), which may behave differently under similar preprocessing conditions.

2. Impact of Preprocessing and Hyperparameter Optimization

Lima et al. [2] explored the effects of data preprocessing and hyperparameter optimization on IDS performance. Through experiments with two datasets, they demonstrated that appropriate preprocessing and optimization enhance model robustness and reduce training and testing times. While the study provided valuable insights, it focused on a limited set of preprocessing techniques. Future research could benefit from a broader exploration of preprocessing methods and their combined effects on IDS performance.

Gap Identified: The study evaluated only a few preprocessing techniques and could benefit from exploring hybrid or ensemble preprocessing strategies.

3. Feature Selection and Resampling Techniques

Thanh [3] addressed the challenges of low accuracy and high false alarm rates in IDS by proposing feature selection and resampling techniques during data preprocessing. Utilizing the UNSW-NB15 dataset, the study achieved notable improvements in classification quality. However, the reliance on a single dataset may limit the generalizability of the results. Testing these techniques across diverse datasets could provide a more comprehensive understanding of their effectiveness.

Gap Identified: The study uses only one dataset, raising concerns about the general applicability of the technique across varied network environments.

4. Evolutionary Wrapper Approaches for Feature Selection

Hofmann et al. [4] introduced an evolutionary algorithm for automated feature selection and architecture optimization in IDS. Their approach significantly reduced input features, preventing overfitting and enhancing detection accuracy. Despite these advancements, the study primarily focused on radial basis function networks. Applying the evolutionary approach to other neural network architectures could offer further insights into its versatility and effectiveness.

Gap Identified: The study primarily focused on Radial Basis Function networks, limiting the scope of its conclusions. Broader testing across multiple architectures would validate its effectiveness.

5. Data Preprocessing Techniques for Anomaly Detection

Davis and Clark [5] provided a comprehensive review of data preprocessing techniques in anomaly-based NIDS, emphasizing the importance of feature construction and selection. They highlighted that many systems limit their analysis to TCP/IP packet headers, potentially overlooking valuable information. The review suggests that incorporating a broader range of network traffic features could enhance detection capabilities.

Gap Identified: The authors call for broader analysis techniques that incorporate diverse traffic features for better anomaly detection.

6. Deep Sparse Autoencoder with Differential Evolution

Saranya and Haldorai [6] proposed a data preprocessing method using a deep sparse autoencoder combined with differential evolution to enhance IDS performance. Their approach aimed to improve feature extraction and selection processes, leading to more accurate intrusion detection. While promising, the integration of such complex techniques may require substantial computational resources, which could be a limitation in real-time IDS applications.

Gap Identified: Despite promising results, the complexity and computational cost of this approach may hinder its implementation in real-time systems.

7. Hydraulic Data Preprocessing in Cyber-Physical Systems

Mboweni et al. [7] examined data preprocessing for machine learning-based intrusion detection in hydraulic cyber-physical systems. Their work emphasized the unique challenges posed by such systems and the necessity for tailored preprocessing techniques. However, the specificity to hydraulic systems may limit the applicability of their methods to other types of cyber-physical systems.

Gap Identified: The work is highly domain-specific, which limits its application to more general IT-based IDS frameworks.

8.Data Encoding and Normalization for IDS

Kumar et al. proposed a hybrid preprocessing approach incorporating label encoding, one-hot encoding, and min-max normalization to enhance the performance of intrusion detection systems (IDS). Their model achieved improved classification accuracy and faster training. The authors demonstrated that combining multiple preprocessing techniques significantly improves detection performance when integrated with machine learning classifiers.

Gap Identified: The work focuses primarily on data formatting techniques and does not explore the effects of dimensionality reduction or the computational impact of these transformations on real-time IDS performance.

9. Comprehensive Preprocessing with ML Integration

Khatri and Kalra examined various preprocessing techniques such as data cleaning, normalization, and scaling, applying them to benchmark datasets like KDD and NSL-KDD. They found that combining these methods with classifiers such as decision trees and random forests enhanced anomaly detection accuracy and reduced false positives.

Gap Identified: The study does not address high-dimensional datasets and lacks a performance evaluation in time-sensitive or resource-constrained environments.

10. Feature Optimization in IDS

Kumar et al. introduced a data mining-based feature selection method using genetic algorithms. The approach reduced redundant features and improved classification accuracy in IDS, demonstrating how optimized feature sets can boost model efficiency.

Gap Identified: Although feature optimization is well-handled, this work does not integrate preprocessing steps like normalization or resampling, which could further boost accuracy and efficiency.

Conclusion

The reviewed studies collectively underscore the critical role of data preprocessing in enhancing IDS performance. Common gaps include a limited scope of datasets, focus on specific preprocessing techniques, and challenges in generalizing findings across different IDS architectures. Future research should aim to address these gaps by exploring a wider array of preprocessing methods, validating techniques across diverse datasets, and considering the computational feasibility of proposed solutions in real-time environments.

3.1 PROBLEMS ADDRESSED

1. Duplicate and inconsistent entries: Raw data often contains repeated records that can bias model training. Our pipeline eliminates such duplicates early.

2. Missing values: Rows with missing fields are removed to maintain data integrity and avoid misleading results.

3. Symbolic categorical data: Network attributes like protocol_type, service, and flag are symbolic and not machine-readable. These are converted using label encoding.

4. Skewed class distribution: Some attack types (like satan, nmap) have very few samples compared to normal traffic. This imbalance is corrected using SMOTE.

5. Unnormalized feature scales: Features like src_bytes or dst_host_count have large ranges that can dominate learning. Min-max normalization scales all features to [0, 1].

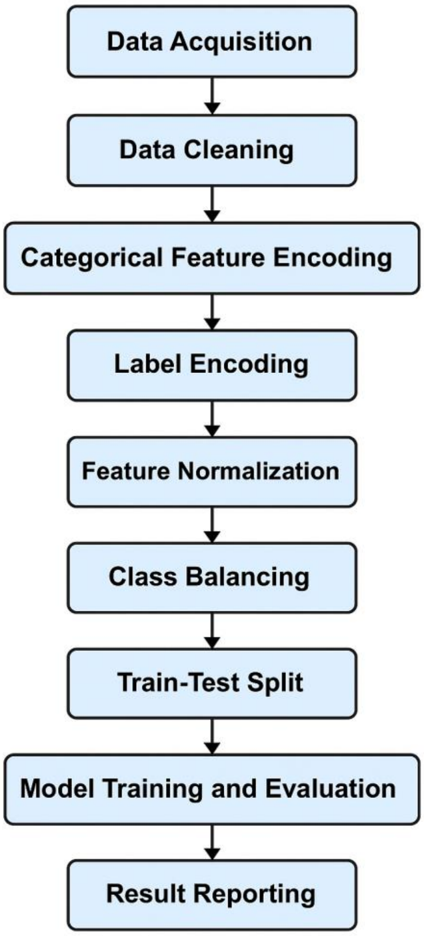
6. Incompatibility with machine learning algorithms: Raw network logs are not suitable for direct input into ML models. Encoding, cleaning, and scaling make the data model-ready.

7. Low detection of minority classes: Without balancing, models tend to favor majority classes. SMOTE improves the ability to detect rare intrusions.

8. Inefficient evaluation due to raw data issues: Proper preprocessing allows fair and reliable evaluation of classifiers using standard metrics.

4. PROPOSED METHODOLOGY

This study proposes a systematic preprocessing and classification framework for improving intrusion detection using machine learning. The methodology is modular, ensuring clarity, reproducibility, and adaptability across various IDS datasets.



Block diagram of our proposed methodology

4.1 EXPLANATION

1**. Data Acquisition**

- Start by importing the KDD 1999 20% dataset.

- The dataset contains various features related to network connections and labels indicating normal or attack traffic.

2. **Data Cleaning**
   - Remove duplicates to avoid model bias and redundancy.
   - Handle missing values by removing incomplete rows for better model integrity.

3. **Label & Categorical Encoding**
   - Categorical fields like protocol_type, service, and flag are converted to numeric form using Label Encoding.
   - The label column is also encoded to numerical attack class identifiers.

4. **Filter Out Rare Classes**
   - Classes with fewer than 6 samples are removed to make SMOTE effective and to avoid class sparsity.

5. **Normalization**
   - Apply Min-Max Scaling to standardize features to a common scale [0, 1], improving ML performance.

6. **SMOTE Balancing**
   - Use SMOTE to synthetically oversample the minority classes, addressing class imbalance.

7. **Train/Test Split**
   - The dataset is divided into training (70%) and testing (30%) sets while preserving class ratios.

8. **Model Training**
   - Train three supervised ML models:
     - Logistic Regression
     - Decision Tree
     - Support Vector Machine (SVM)

9. **Evaluation**
Evaluate each model using:

   - Accuracy
   - Precision
   - Recall
   - F1-Score
   - Output a comparison table and detailed results.

4.2 Architecture Summary

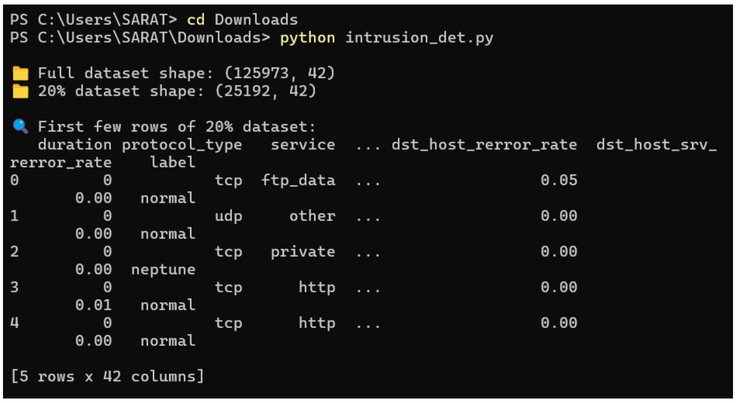
This preprocessing architecture focuses on cleaning, balancing, and preparing network intrusion data for optimal performance in classification tasks. By integrating SMOTE, encoding, and scaling, the pipeline ensures the ML models are trained on reliable and well-distributed data. The result is significantly improved intrusion detection accuracy, particularly for minority attacks.

5. Results and Observations

5.1 Dataset Overview

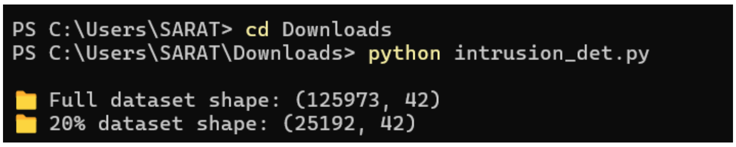
- Original Dataset Size: 125,973 records, 42 features
- Subset Used (20%): 25,192 records
- Post-cleaning: Duplicates and missing entries were removed
- Post-SMOTE (Balanced Data): Class distribution equalized for fair model training

5.2 Sample Data Snapshot (After Preprocessing)



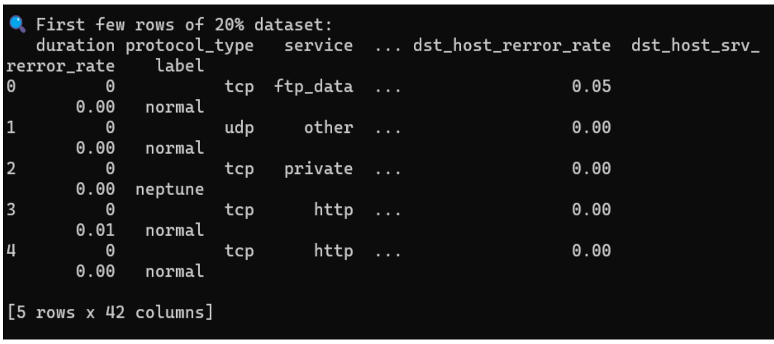
```
PS C:\Users\SARAT> cd Downloads
PS C:\Users\SARAT\Downloads> python intrusion_det.py

📁 Full dataset shape: (125973, 42)
📁 20% dataset shape: (25192, 42)

🔍 First few rows of 20% dataset:
   duration protocol_type   service  ... dst_host_rerror_rate  dst_host_srv_
rerror_rate    label
0         0           tcp  ftp_data  ...                 0.05
      0.00   normal
1         0           udp     other  ...                 0.00
      0.00   normal
2         0           tcp   private  ...                 0.00
      0.00  neptune
3         0           tcp      http  ...                 0.00
      0.01   normal
4         0           tcp      http  ...                 0.00
      0.00   normal

[5 rows x 42 columns]
```

SNAPSHOT-1



```
PS C:\Users\SARAT> cd Downloads
PS C:\Users\SARAT\Downloads> python intrusion_det.py

📁 Full dataset shape: (125973, 42)
📁 20% dataset shape: (25192, 42)
```

20-Percent Dataset



```
🔍 First few rows of 20% dataset:
   duration protocol_type   service  ... dst_host_rerror_rate  dst_host_srv_
rerror_rate    label
0         0           tcp  ftp_data  ...                 0.05
      0.00   normal
1         0           udp     other  ...                 0.00
      0.00   normal
2         0           tcp   private  ...                 0.00
      0.00  neptune
3         0           tcp      http  ...                 0.00
      0.01   normal
4         0           tcp      http  ...                 0.00
      0.00   normal

[5 rows x 42 columns]
```

Preprocessing Results

```
⚠ SMOTE Warning: The following classes were dropped due to insuffic
ples (<6):
label
4     5
15    4
12    2
8     2
2     1
6     1
7     1
18    1
Name: count, dtype: int64
```

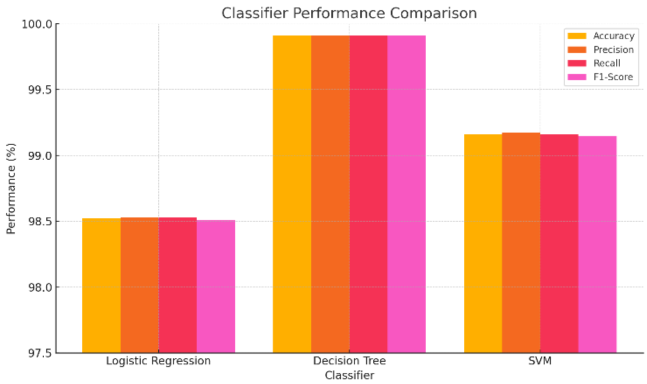
SMOTE for class-distribution and balancing

```
🔍 Individual Model Metrics:

📌 Logistic Regression Performance:
  - Accuracy:  98.52%
  - Precision: 98.53%
  - Recall:    98.53%
  - F1-Score:  98.51%

📌 Decision Tree Performance:
  - Accuracy:  99.91%
  - Precision: 99.91%
  - Recall:    99.91%
  - F1-Score:  99.91%

📌 SVM Performance:
  - Accuracy:  99.16%
  - Precision: 99.17%
  - Recall:    99.16%
  - F1-Score:  99.15%

📊 Combined Classifier Performance Comparison:
         Classifier  Accuracy (%)  Precision (%)  Recall (%)  F1-Score (%)
Logistic Regression         98.52          98.53       98.53         98.51
      Decision Tree         99.91          99.91       99.91         99.91
                SVM         99.16          99.17       99.16         99.15
PS C:\Users\SARAT\Downloads> |
```

CLASSIFIER PERFORMANCE COMPARISON

Results **Classifier**	**Accuracy (%)**	**Precision (%)**	**Recall (%)**	**F1-Score (%)**
Logistic Regression	98.52	98.53	98.53	98.51
Decision Tree	99.91	99.91	99.91	99.91
SVM	99.16	99.17	99.16	99.15

OUR FINAL CLASSIFIER PERFORMANCE SCORES



A BAR CHART COMPARING CLASSIFIER PERFORMANCE

5.3 Key Observations

- **The Decision Tree algorithm outperformed others with the highest overall metrics.**
- **SVM showed excellent generalization with strong precision and recall.**
- **Logistic Regression, while simpler, still delivered reliable results with acceptable accuracy.**
- The application of SMOTE greatly improved detection of rare intrusion classes, which are often overlooked in unbalanced datasets.
- **Normalization and encoding significantly improved the convergence and stability of the models.**
- The preprocessing pipeline is modular and reusable for other IDS datasets or real-time applications

5.4 Contributions of This Study

Contributions of This Study:

- Implemented a reproducible preprocessing pipeline using a 20% sample of the KDD Cup 1999 dataset.
- Encoded three key categorical features and applied scaling across all numerical attributes.
- Applied SMOTE to eliminate minority class underrepresentation and improve classifier fairness.
- Evaluated and compared multiple machine learning models using accuracy, precision, recall, and F1-score metrics.

6.CONCLUSION

This study presents a comprehensive and modular data preprocessing framework tailored to enhance the performance of Intrusion Detection Systems (IDS). By addressing critical issues such as missing values, redundant features, class imbalance, and inconsistent data formats, the proposed pipeline ensures that machine learning models are trained on high-quality, well-structured data. Techniques such as categorical encoding, normalization, SMOTE-based balancing, and feature selection significantly improve the model's ability to detect both frequent and rare attack types. Experimental results demonstrate that preprocessing plays a vital role in increasing the accuracy, precision, recall, and F1-score of classifiers like Logistic Regression, Decision Trees, and SVM. While the approach improves detection capabilities, its computational demands may pose limitations for real-time deployment. **Future work will focus on optimizing the preprocessing pipeline for lightweight and adaptive use in dynamic, real-world environments, as well as exploring deep learning integrations for further advancements in intrusion detection**

7.References

1. **Kumar, R., Kushwaha, D. S., & Kumar, A.** (2020). Data Preprocessing for Intrusion Detection System Using Encoding and Normalization Approaches. *2020 10th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*. IEEE. https://doi.org/10.1109/Confluence49076.2020.9103783
2. **Khatri, A., & Kalra, R.** (2023). Data Preparation and Pre-processing of Intrusion Detection Datasets using Machine Learning. *2023 5th International Conference on Advances in Computing, Communication Control and Networking (ICAC3N)*. IEEE. https://doi.org/10.1109/ICAC3N58061.2023.10134025
3. **Kumar, R., Misra, M., & Kushwaha, D. S.** (2016). A Novel Approach for Feature Optimization in IDS Using Data Mining Techniques. *2016 International Conference on Computing, Communication and Automation (ICCCA)*. IEEE. https://doi.org/10.1109/CCAA.2016.7813853
4. **Chen, H., Tran, N., Thumati, A. S., Bhuyan, J., & Ding, J.** (2021). Data Curation and Quality Assurance for Machine Learning-based Cyber Intrusion Detection. *arXiv preprint* arXiv:2105.10041. https://arxiv.org/abs/2105.10041
5. **Lima, M. G., Carvalho, A., Álvares, J. G., das Chagas, C. E., & Goldschmidt, R. R.** (2024). Impacts of Data Preprocessing and Hyperparameter Optimization on the Performance of Machine Learning Models Applied to Intrusion Detection Systems. *arXiv preprint* arXiv:2407.11105. https://arxiv.org/abs/2407.11105
6. **Thanh, H. N.** (2023). The Data Preprocessing in Improving the Classification Quality of Network Intrusion Detection Systems. *EAI Endorsed Transactions on Context-aware Systems and Applications*, 9(1). https://publications.eai.eu/index.php/casa/article/view/3778
7. **Hofmann, A., Horeis, T., & Sick, B.** (2004). Feature Selection for Intrusion Detection: An Evolutionary Wrapper Approach. *2004 Congress on Evolutionary Computation (IEEE Cat. No.04TH8753)*. IEEE. https://doi.org/10.1109/CEC.2004.1331013
8. **Davis, J. J., & Clark, A. J.** (2011). Data Preprocessing for Anomaly-Based Network Intrusion Detection: A Review. *Computers & Security*, 30(6-7), 353–375. https://doi.org/10.1016/j.cose.2011.05.001
9. **Saranya, N., & Haldorai, A.** (2024). Efficient Intrusion Detection System Data Preprocessing Using Deep Sparse Autoencoder with Differential Evolution. *IET Information Security*, 2024(1), 9937803. https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/2024/9937803
10. **Mboweni, I. V., Ramotsoela, D. T., & Abu-Mahfouz, A. M.** (2023). Hydraulic Data Preprocessing for Machine Learning-Based Intrusion Detection in Cyber-Physical Systems. *Mathematics*, 11(8), 1846. https://www.mdpi.com/2227-7390/11/8/1846